

Designing a RISC-V SoC

Course material for subject Digital Design

Ratko Pilipović

2026-10-01

Table of contents

Preface	5
1 Introduction to Digital Design	6
Introduction to Digital Design	7
1.1 Goal for this week	7
1.2 The P-N junction	7
1.2.1 The depletion region	9
1.3 CMOS technology	10
1.3.1 MOSFET transistors	11
1.3.2 Complementary MOS	14
1.4 CMOS characteristics	15
1.4.1 Transfer function	15
1.4.2 Noise margins	16
1.4.3 Power	16
1.4.4 Dynamic behavior	17
1.5 Manufacturing CMOS integrated circuits	17
1.5.1 Photolithography process	18
2 Basics of Digital Integrated Circuits	19
Basics of Digital Integrated Circuits	20
2.1 Goal for this week	20
2.2 Digital integrated circuits	20
2.2.1 Scale of integration	20
2.3 Design methodologies	21
2.3.1 Standard integrated circuits	21
2.3.2 Application-specific integrated circuits	21
2.3.3 Building blocks for ASIC design	23
2.4 Standard ASICs (SASIC)	24
2.4.1 Configuration technologies	24
2.4.2 Simple programmable logic devices	25
2.4.3 Complex programmable logic devices	26
3 Introduction and Review of Fundamentals	28

Introduction	29
4 FPGA Fundamentals	30
FPGA Fundamentals	31
4.1 Goal for this week	31
4.2 Why reconfigurable hardware?	31
4.3 The three building blocks	33
4.3.1 Configurable Logic Block (CLB)	34
4.3.2 Routing plane: programmable interconnect	37
4.3.3 I/O blocks	38
4.3.4 Reconfigurability, level by level	38
4.4 Case study: Xilinx Series-7	39
4.4.1 CLB: SLICEL and SLICEM	40
4.4.2 Block RAM and DSP slices	42
4.4.3 Clock management and interconnect	42
4.5 FPGA design flow	43
5 SystemVerilog — Syntax and Synthesizable Coding Style	45
SystemVerilog basics	46
5.1 Goal for this week	46
5.2 It looks like C. It isn't.	46
5.3 Signals vs. variables	46
5.4 Combinational logic: always_comb	47
5.5 Sequential logic: always_ff	47
5.6 Reset styles	48
5.7 From syntax to the pipeline	48
6 Five-Stage Pipeline — Structure	49
Five-stage pipeline	50
6.1 Goal for this week	50
6.2 Modules for this week	50
6.3 Example: pipeline_pkg.sv	51
6.3.1 Why a struct instead of individual signals	51
6.4 Next step	52
7 Five-Stage Pipeline — Hazards	53
Pipeline hazards	54
8 Memory — Implementation in SystemVerilog	55

Memory	56
9 Verification with CocoTB	57
Verification with CocoTB	58
References	59

Preface

This material was written alongside the lectures for the course DN, redesigned around the theme of RISC-V SoC design. Each chapter corresponds to one week of lectures and serves a dual purpose:

1. As **lecture notes** — rendered as a slide deck (reveal.js).
2. As a **book chapter** — rendered as part of the combined PDF/HTML course material.

Content intended for only one of the two formats (e.g. discussion questions used in class, or in-depth derivations meant for the book) is marked with `content-visible` blocks — see the example in the chapter on pipeline structure.

Work in progress

This material is a work in progress. It is written and revised alongside the course as it is taught, so chapters vary in polish, and some sections still need further revision. Please read it with that in mind, and feel free to flag anything that is unclear, incomplete, or incorrect.

1 Introduction to Digital Design

Introduction to Digital Design

1.1 Goal for this week

- See why the P-N junction's one-way conduction is the basic building block behind every diode and every transistor
- Follow how a MOSFET's gate voltage creates, and destroys, a conducting channel between source and drain, turning it into a voltage-controlled switch
- Build the CMOS inverter from one NMOS and one PMOS, and see why that pairing is what gives CMOS its low static power
- Separate static and dynamic power, and see which design choices trade one for the other
- Connect the transistor-level picture to the chip-level one: how NMOS and PMOS actually end up side by side on the same piece of silicon, and roughly how a wafer becomes a chip

1.2 The P-N junction

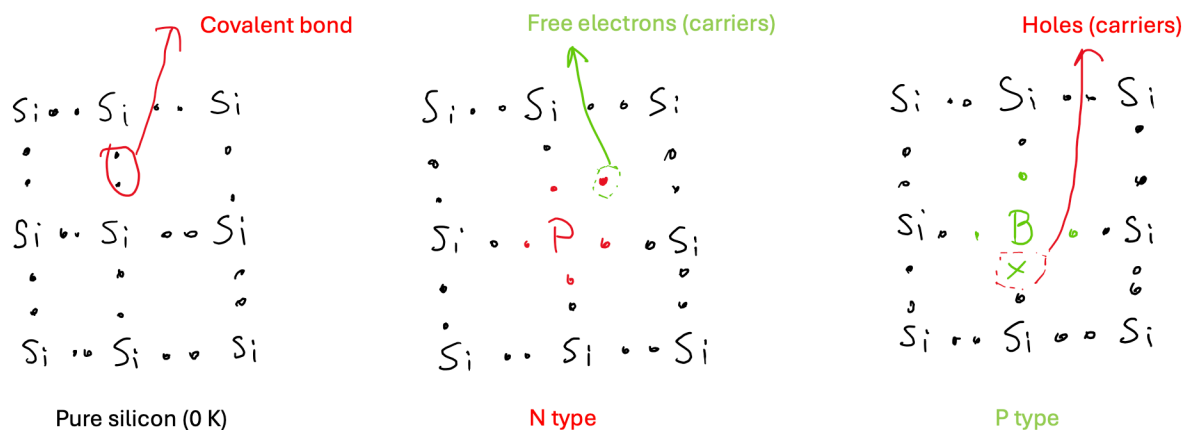


Figure 1.1: N-type and P-type silicon: doping replaces some silicon atoms with impurities that donate either a free electron or a hole

One of the fundamental facts of solid-state electronics is how a P-N junction behaves. On its own, pure silicon is a poor conductor. We fix that through doping, deliberately introducing a small number of impurity atoms into the silicon lattice to change how many free charge carriers it has. Dope silicon with an element that has more valence electrons than silicon, such as phosphorus or arsenic, and the extra electrons become free carriers. We call the result n-type silicon, “n” for the negative charge of its majority carrier. Dope it instead with an element that has fewer valence electrons, such as boron or gallium, and the missing electron leaves behind a “hole”, a position that behaves like a mobile positive charge. We call that p-type silicon.

Bring a block of n-type and a block of p-type silicon into contact, and the junction between them behaves in a way neither material does alone. Current flows easily in one direction, forward bias, and is almost completely blocked in the other, reverse bias. This one-way behavior is not a separate property we add. It falls directly out of how charge moves across the junction, and it is the basic mechanism behind every diode, and in turn, every transistor we build in this course.

1.2.1 The depletion region

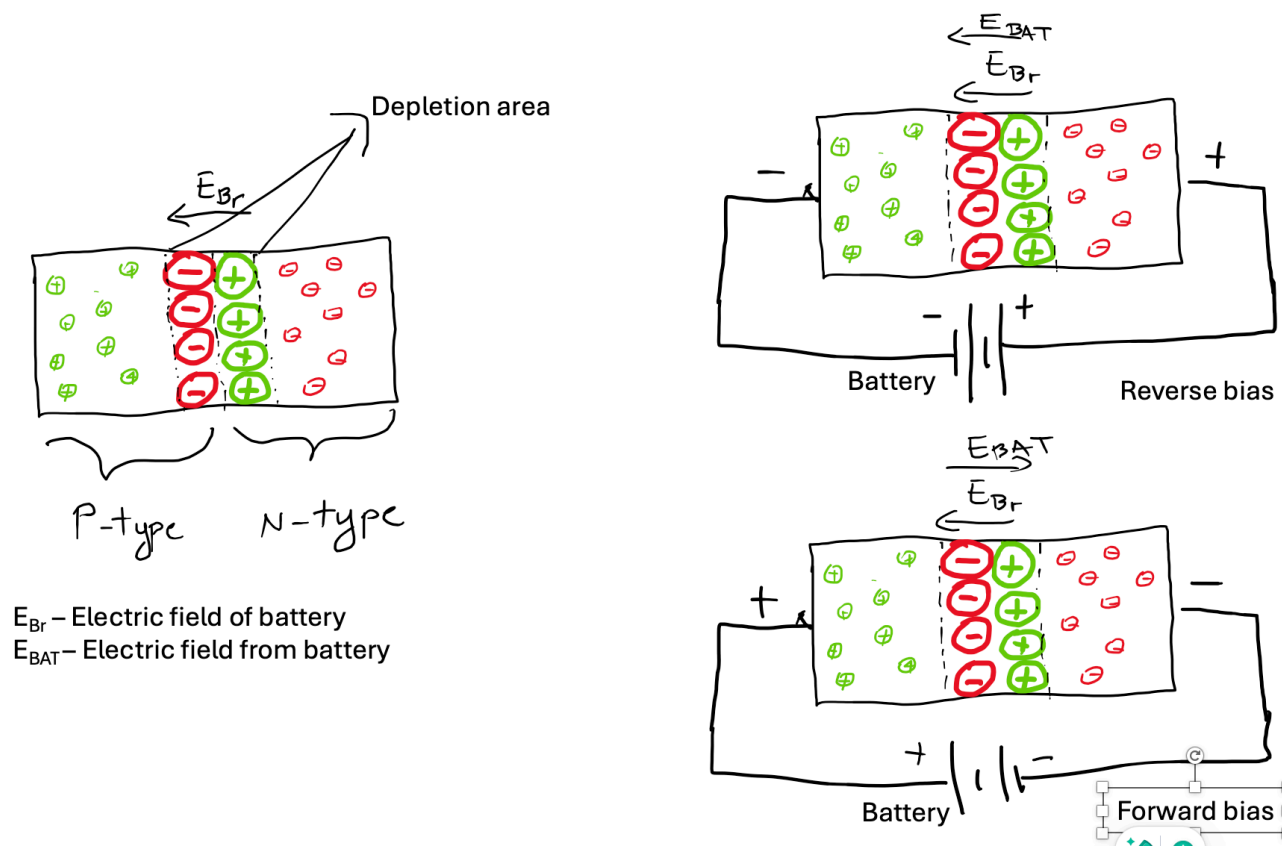


Figure 1.2: The depletion region at a P-N junction, and how reverse bias and forward bias each move the electric field relative to the battery's field

Right at the junction, free electrons from the n-side and free holes from the p-side meet and recombine, leaving behind a thin region with no free carriers at all. We call this the depletion region. What is left behind there is not neutral. It is the fixed, immobile dopant ions that donated those carriers in the first place, and those fixed charges set up an electric field across the depletion region, pointing from the n-side toward the p-side.

1.2.1.1 Equilibrium

With no external voltage applied, that built-in field reaches a balance point. It is strong enough to stop any further electrons or holes from diffusing across the junction, but no stronger. We call this the equilibrium state. Electrically, it behaves like a small built-in

barrier, often called the Coulomb barrier, that a carrier has to overcome before it can cross from one side to the other.

1.2.1.2 Reverse bias

Apply an external voltage with its positive terminal on the n-side, and that external field points the same way as the junction's built-in field. The two add together, the depletion region widens, and the barrier an electron would need to cross gets taller, not shorter. For current to flow at all, electrons from the n-side would have to reach the junction and recombine with holes on the p-side, and a reverse-biased junction actively drives electrons away from the junction instead. The result is that almost no current flows.

1.2.1.3 Forward bias

Flip the polarity, and the external field now opposes the built-in field instead of reinforcing it. The depletion region shrinks, the Coulomb barrier collapses, and carriers can cross the junction with very little resistance. This is forward bias, and it is the conducting state of a diode.

1.3 CMOS technology

Complementary Metal-Oxide-Semiconductor (CMOS) technology is, practically speaking, unavoidable in digital design. It builds every logic function from paired n-type and p-type MOSFETs, and that pairing is what lets it combine very low static power with high noise immunity and fast switching, all at once. That combination is exactly what large-scale integration needs, whether that means a battery-powered microcontroller or the millions of logic gates on a modern microprocessor. Every digital building block we design in this course, down to the CLB we met in the FPGA fabric, ultimately bottoms out in CMOS transistors like the ones in this chapter.

1.3.1 MOSFET transistors

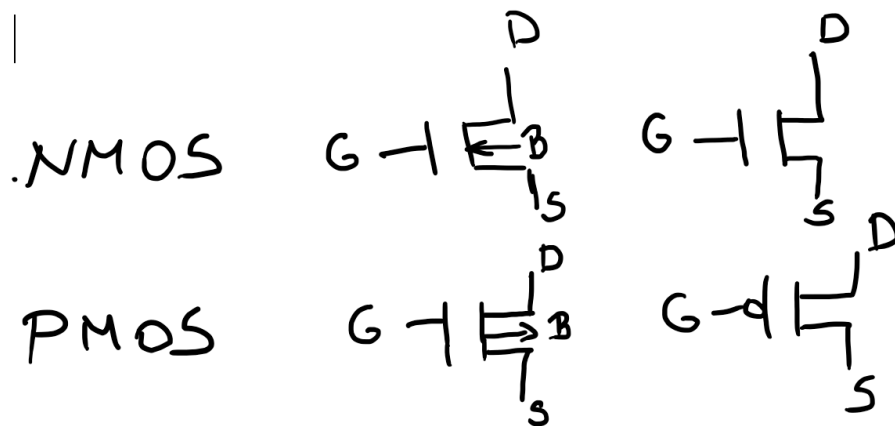


Figure 1.3: NMOS and PMOS schematic symbols: the four-terminal version shows the body connection explicitly, the three-terminal version assumes it

We will mostly use the MOSFET as a switch, though it can also amplify signals. At first glance it looks like a four-terminal device: gate, source, drain, and body. A voltage on the gate decides whether, and how strongly, current can flow between source and drain. The body terminal shifts the device's voltage-current characteristics and is normally just tied to a fixed supply rail, which is why many schematic symbols leave it out and show only three terminals.

The name spells out what the device is built from. "Metal-Oxide-Semiconductor" describes its physical structure. "Field-Effect Transistor" describes how it works: an electric field, not a direct physical connection, controls the current.

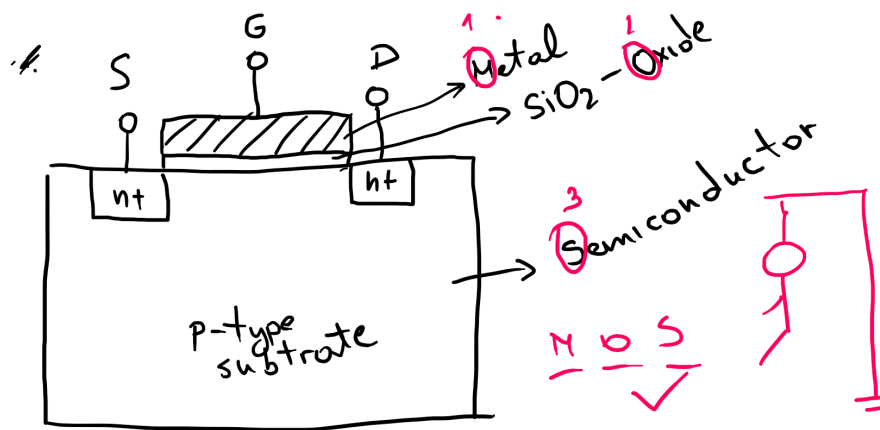


Figure 1.4: NMOS cross-section: a metal gate over an oxide insulator, sitting above a p-type substrate with two n+ regions for source and drain

We illustrate an NMOS. Its gate is a metal layer separated from the p-type substrate beneath it by a thin oxide layer, which insulates the gate and prevents any DC current from flowing into it. Source and drain are both n+ regions embedded directly in that p-type substrate. The names describe their role in conduction: the source supplies the charge carriers, electrons for an n-channel device and holes for a p-channel one, that flow through the channel, and the drain is where those carriers leave it.

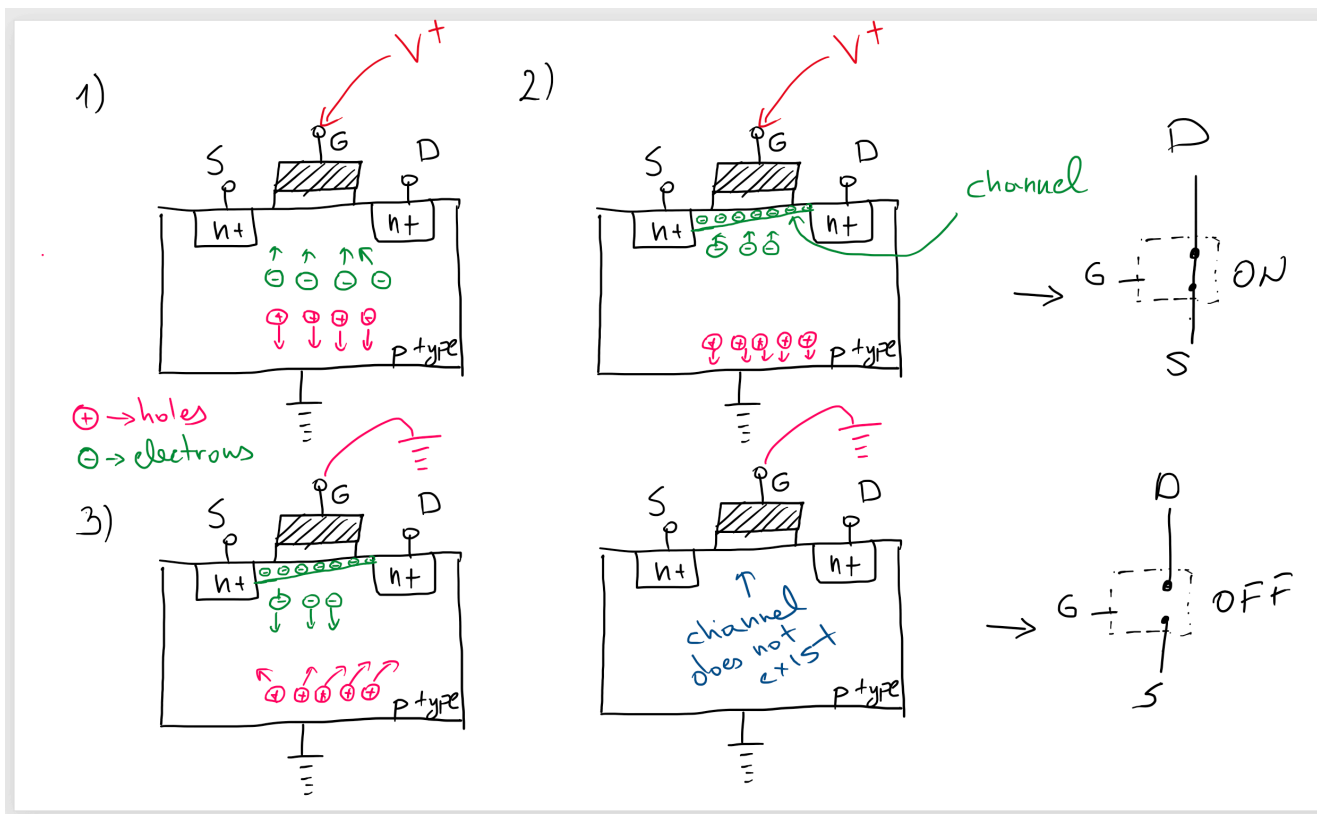


Figure 1.5: Turning an NMOS on: a positive gate voltage pulls electrons to the surface and forms a conducting channel between source and drain, which collapses again once the gate voltage is removed

To turn an NMOS on, we apply a positive voltage to the gate. That voltage attracts electrons from the p-type substrate toward the surface and pushes holes downward and away, purely through the electric field it creates, which is exactly what makes this a field-effect transistor. Once enough electrons have gathered at the surface, they form a bridge, the channel, connecting source and drain, and current can flow between them. How long that channel is, geometrically, is a property of the fabrication technology used.

In effect, the MOS transistor behaves as a controlled switch. Take the voltage off the gate, and the electrons no longer have anywhere to stay. The channel dissolves, and the connection between drain and source opens back up.

1.3.2 Complementary MOS

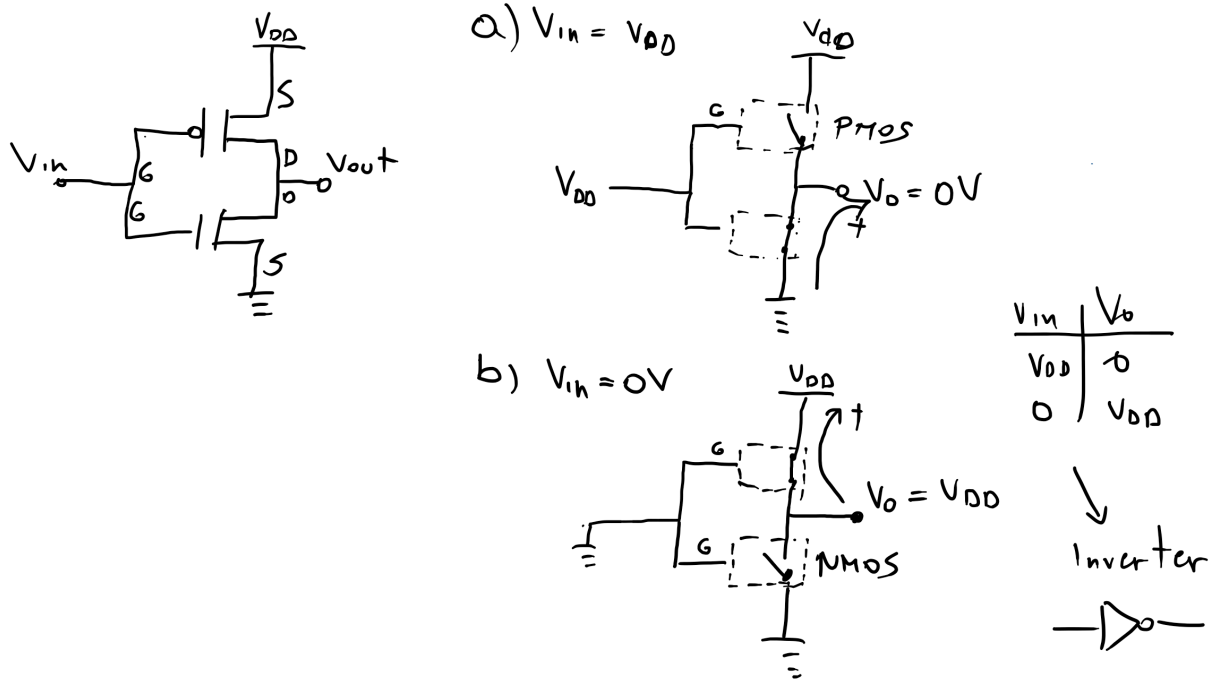


Figure 1.6: CMOS inverter: tying NMOS and PMOS gates together means exactly one of them conducts for any input, pulling the output to either ground or VDD

A CMOS gate pairs an NMOS and a PMOS transistor that share a common gate input. We already know how to turn an NMOS on, and a PMOS is its mirror image: it turns on when the gate is low rather than high. Tie a positive voltage to the shared gate, and the NMOS turns on while the PMOS turns off, pulling the output down to ground. Tie the gate to ground instead, and the PMOS turns on while the NMOS turns off, pulling the output up to VDD. In both cases, exactly one of the two transistors conducts, never both, and the output ends up at the opposite logic level from the input. That is an inverter, and it is the simplest possible CMOS gate.

1.4 CMOS characteristics

1.4.1 Transfer function

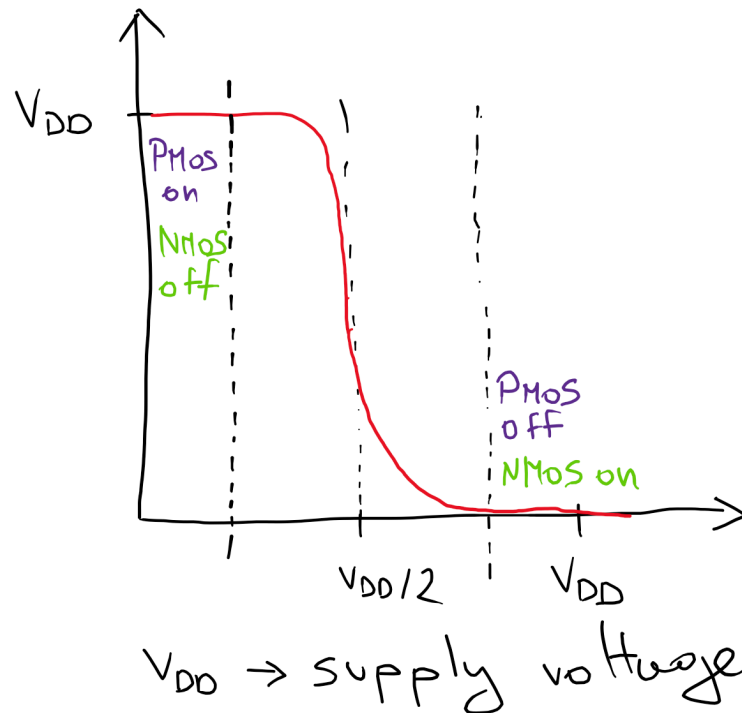


Figure 1.7: CMOS inverter transfer function: the output stays near V_{DD} or near ground across most of the input range, and switches abruptly near $V_{DD}/2$

The transfer function shows how the output voltage depends on the input voltage. For a CMOS inverter, it is almost a step function. For a wide range of low inputs, the PMOS is on and the NMOS is off, and the output sits close to V_{DD} . For a wide range of high inputs, the roles reverse and the output sits close to ground. Only in a narrow band around $V_{DD}/2$ do both transistors conduct at once, and the output swings sharply from one rail to the other. That sharp transition is exactly what we want from a digital gate. It is what keeps a “1” looking like a clean 1 and a “0” looking like a clean 0, right up until the input crosses the switching threshold.

1.4.2 Noise margins

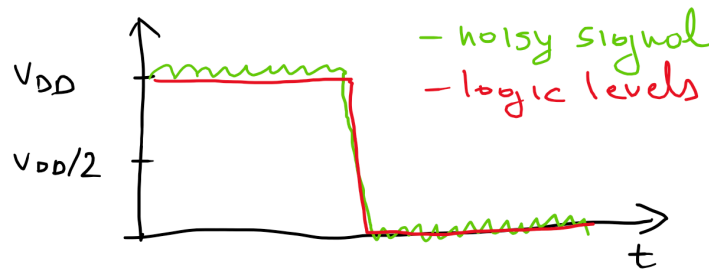


Figure 1.8: A noisy signal riding on a CMOS logic level. As long as the noise stays within the flat regions of the transfer function, it never gets interpreted as the wrong logic level

Because the transfer function stays flat near V_{DD} and near ground across most of the input range, a CMOS gate tolerates a surprising amount of noise. A signal can pick up noise from neighboring wires or supply fluctuations and still be read correctly, as long as that noise does not push the voltage past the switching threshold in the middle of the transfer function. This margin between “close enough to a clean logic level” and “close enough to flip the gate’s decision” is the noise margin, and CMOS’s wide noise margins are one of the main reasons it dominates digital design.

1.4.3 Power

CMOS technology is known for its low power consumption, which is one of its biggest advantages over older logic families. We split that power consumption into two categories: static power and dynamic power.

Static power (also called leakage power) is what the circuit consumes while it is not switching at all. It comes mostly from small leakage currents through transistors that are nominally off, and it becomes a much bigger concern as transistor geometries shrink.

Dynamic power is what the circuit consumes while it is actively switching. Every time a gate’s output toggles, we have to charge or discharge its load capacitance, and that costs energy. We can express the dynamic power of a single node as

$$P_{dynamic} = \alpha CV^2 f$$

where α is the activity factor (the fraction of clock cycles in which the node actually switches), C is the load capacitance, V is the supply voltage, and f is the switching frequency.

1.4.4 Dynamic behavior

A CMOS gate does not switch instantaneously. To change its output, it first has to charge the load capacitance, when the output rises, or discharge it, when the output falls. That load capacitance is not one single thing. It is the sum of the input capacitance of every gate the output drives, the capacitance of the interconnect wiring between them, and the capacitance of the reverse-biased P-N junctions inside the transistors themselves. To make a gate switch faster, we have a few levers: raise the supply voltage, widen the transistor's channel width-to-length ratio, or simply drive fewer downstream gates from the same output.

1.5 Manufacturing CMOS integrated circuits

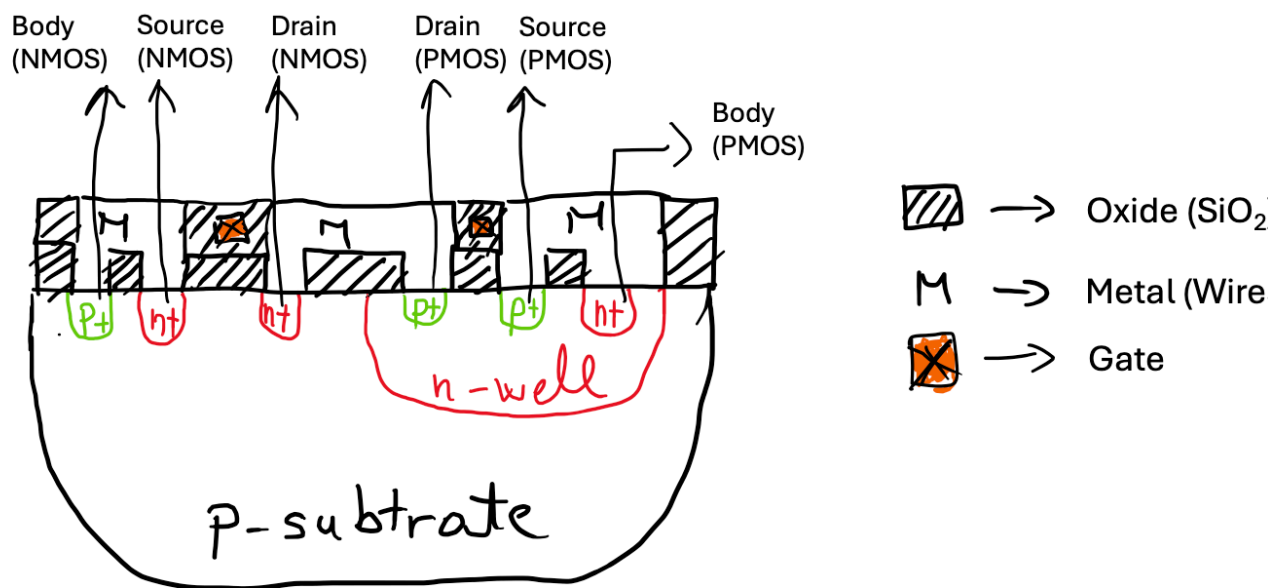


Figure 1.9: Cross-section of a CMOS chip: the PMOS sits inside an n-well so it can share the same p-type substrate as the NMOS

Building CMOS means putting NMOS and PMOS transistors on the very same piece of silicon, even though an NMOS wants a p-type substrate and a PMOS wants an n-type one. The usual solution is an n-well: a pocket of n-type silicon diffused into the p-type substrate, deep enough to host a complete PMOS transistor, source, drain, and body, while the NMOS transistors around it sit directly in the surrounding p-type substrate as before. Everything above that, the oxide layers and the metal wiring that connects the transistors into actual logic gates, is built in the same set of process steps for both transistor types.

1.5.1 Photolithography process

Turning a bare silicon wafer into a working chip is a repeated pattern of deposit, pattern, and etch, applied layer by layer. It starts with the wafer itself, onto which we grow a thin layer of silicon dioxide (SiO_2) through oxidation. On top of that, we spin on a photoresist, a light-sensitive polymer. By default we use a negative resist, which becomes soluble wherever light hits it, though a positive resist, which behaves the opposite way, is also an option depending on the process.

Next comes the stepper exposure. A glass mask carrying the pattern for this layer is brought close to the wafer. Its opaque regions block UV light, and its transparent regions let it through, exposing the resist beneath in exactly the shape we want to process. After exposure, we develop and bake the wafer: an acid or base solution washes away the soluble part of the resist, and baking hardens what is left into a solid protective mask.

With that mask in place, acid etching removes material, typically the oxide layer, wherever the resist is not protecting it. A spin, rinse, and dry step cleans the wafer before whatever comes next, whether that is another masked layer or a different process step entirely. Once a layer is finished, we strip away the remaining photoresist, and the wafer is ready for the next pass through this same cycle, one layer at a time, until the full set of transistors and interconnect is complete.

2 Basics of Digital Integrated Circuits

Basics of Digital Integrated Circuits

2.1 Goal for this week

- Classify integrated circuits by fabrication technology (bipolar, MOS/CMOS, BiCMOS) and by scale of integration, and see why Moore's law is the reason those scales keep climbing
- Separate the IC business models: standard parts built for nobody in particular, full-custom ASICs built for one customer, and the semi-custom middle ground
- See how standard cells, macrocells, and gate arrays let an ASIC designer reuse pre-built blocks instead of hand-laying every transistor
- Meet the predecessors to the FPGA: SPLDs (PAL, PLA, PLE) and CPLDs, and the handful of underlying technologies a chip can use to store its configuration

2.2 Digital integrated circuits

A digital integrated circuit (IC) is, in simple terms, a miniature, low-cost circuit made of active and passive elements, all fabricated together on a single silicon die. We can classify ICs by the transistor technology they are built from: bipolar junction transistors (BJT), unipolar MOS or CMOS transistors like the ones we met in the previous chapter, or a mix of both, called BiCMOS.

Bipolar junction transistors have real advantages over CMOS. They switch at higher frequencies, offer better gain, and can drive higher output currents. CMOS wins anyway for almost all digital design, because a BJT's static power consumption is never quite zero, and BJTs are larger and slower to fabricate at a given density. That is exactly the low-static-power, high-density tradeoff that made CMOS the default technology in the previous chapter.

2.2.1 Scale of integration

We also classify ICs by their scale of integration, a rough measure of how many basic elements, transistors or gates, fit on one chip. Small scale integration (SSI) covers up to about 100 elements, medium scale (MSI) 100 to 1,000, large scale (LSI) 1,000 to 10,000, very

large scale (VLSI) 10,000 to 100,000, and ultra large scale (ULSI) 100,000 to 1,000,000. Beyond that, wafer-scale integration treats an entire silicon wafer as a single circuit rather than cutting it into individual dies.

This progression is not just a naming exercise. Gordon Moore observed in 1965 that the number of transistors we can fit on a chip roughly doubles every two years, a trend we now call Moore's law. It is the reason the scale of integration climbed from a handful of gates per chip in the 1960s to transistor counts in the billions today, and it is the main force behind nearly every capability we take for granted in modern digital design.

2.3 Design methodologies

We can also classify digital ICs by who they are designed for, which we call their design methodology: standard ICs (SIC), application-specific ICs (ASIC), and standard ASICs (SA-SIC).

2.3.1 Standard integrated circuits

A standard IC is designed for an unknown customer. Whoever eventually buys the chip has no influence on how it was designed, and that is exactly what lets it be mass-produced and sold cheaply.

The earliest standard ICs were simple, fixed-function parts, and building any larger system out of them meant wiring together many of them on a board to form whatever circuit was needed. Software-programmable standard parts changed that, above all the CPU: the same piece of hardware adapts to a new task just by changing the program it runs, instead of changing its wiring. That flexibility is not free. A general-purpose CPU is rarely the most efficient way to run any one specific task, a tradeoff we look at more closely when we get to FPGAs later in this book.

2.3.2 Application-specific integrated circuits

An application-specific integrated circuit (ASIC) is designed for one specific application or field, rather than for an unknown general-purpose market. We build ASICs in two ways: full-custom and semi-custom.

2.3.2.1 Full-custom design

A full-custom design is built specifically for one customer, with every mask layer needed to fabricate the chip designed to order for that customer's exact need, whether that is a systolic array, genome analysis hardware, or a dedicated accelerator. The design itself is typically strongly confidential. Since the design cost is comparable to a standard IC's but spread over a far smaller production run, full-custom chips carry a high price per unit.

2.3.2.2 Semi-custom design

Semi-custom design starts from a predesigned or preprocessed chip. Only certain mask layers get adapted to the customer's needs, while the rest stay identical across every customer who buys that base design. The manufacturer keeps a library of pre-tested modules on hand and connects them to fit each customer's requirements, rather than designing from scratch every time. Because the same base design is reused across many customers, semi-custom chips are much cheaper per unit than full-custom ones.

2.3.3 Building blocks for ASIC design

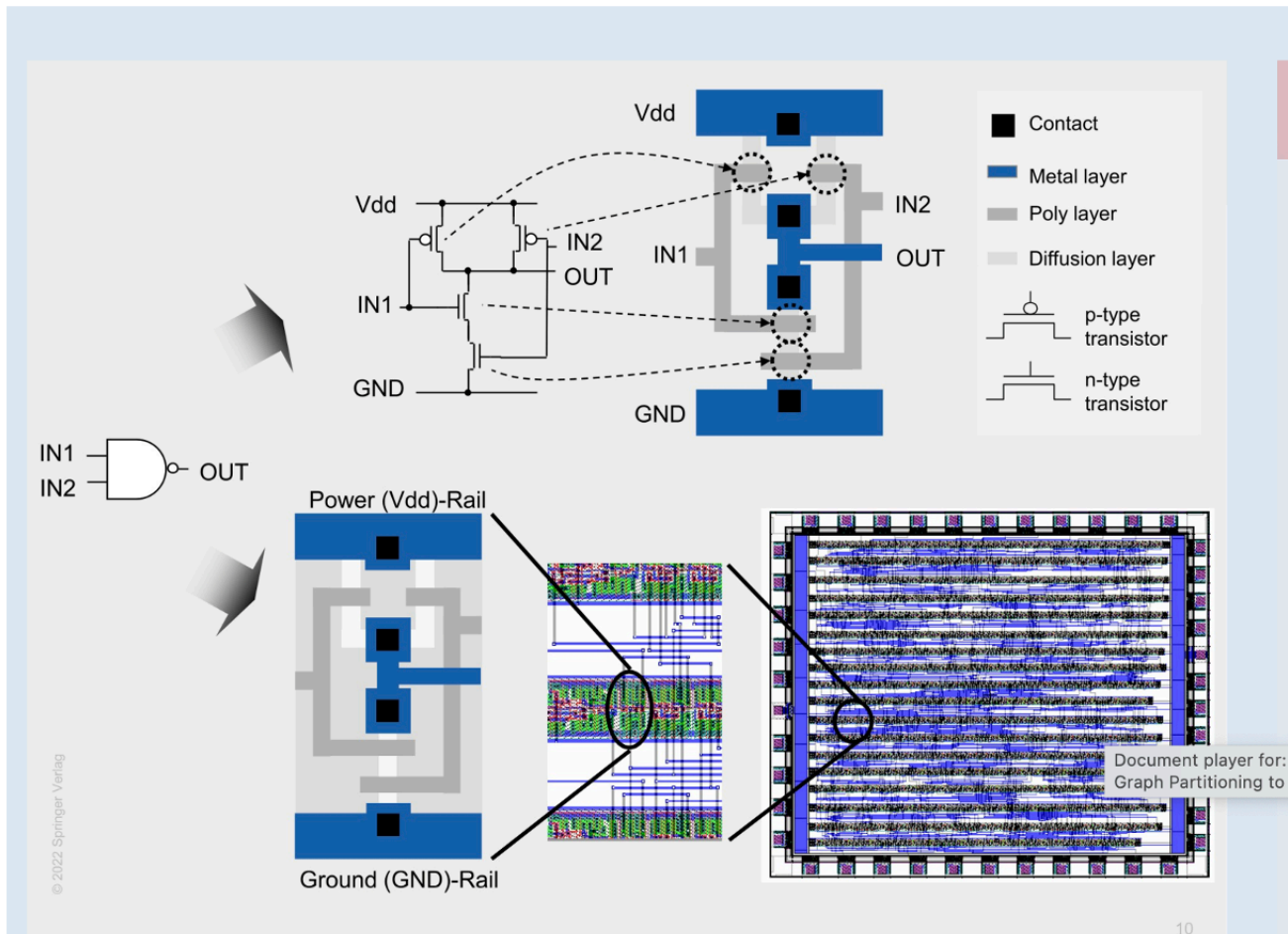


Figure 2.1: Standard-cell design flow: a NAND2 gate as a transistor schematic, its layout in metal, poly, and diffusion layers, and the resulting row-based placement on a die (source: Kahng et al., *VLSI Physical Design: From Graph Partitioning to Timing Closure*, © 2022 Springer Verlag)

Semi-custom and standard-cell ASIC design rests on a few kinds of reusable building block.

Standard cells are small, universal blocks, logic gates, flip-flops, multiplexers, and the like, each with a predefined layout and electrical behavior. The figure above shows a two-input NAND gate as a standard cell: its transistor schematic, its layout in metal, polysilicon, and

diffusion layers, and the fixed-height block that results, ready to sit in a row next to other cells. Every standard cell in a library shares that same row height, which is what lets thousands of them be tiled into rows and wired together with routing that runs above the cells, so-called over-the-cell routing. A chip's entire library of standard cells, together with the data needed to use them, is called a process design kit (PDK).

Macrocells and megacells are a step up from standard cells: larger, pre-laid-out blocks such as adders, multipliers, registers, or even whole microprocessors. The difference between the two is who they were designed for. A megacell is designed without knowledge of the specific application that will use it, which makes it broadly reusable, while a macrocell is designed to order for one customer's needs.

Gate arrays take a different approach. Their basic elements, individual gates or groups of unconnected transistors, are pre-fabricated across the whole die before any customer design is known. Implementing a specific design on a gate array is then purely a metal-wiring problem: connect the gates that are already there in whatever pattern the design needs. Early gate arrays separated these blocks with dedicated routing channels. Later "sea of gates" designs packed transistors edge to edge instead and used higher-level metal layers for routing, improving density, with connections made by shorting or bridging transistors that would otherwise stay unconnected.

2.4 Standard ASICs (SASIC)

A standard ASIC (SASIC), also called a configurable ASIC, is manufactured generically, the same way a standard IC is, and only afterward configured to behave like a custom one. What changes from one application to the next is not the silicon itself. It is a set of configuration bits, held in one of a handful of underlying technologies.

2.4.1 Configuration technologies

Static memory is the technology we will see the most of later in this book. Configuration bits live in SRAM cells and drive electronic switches (transmission gates, tri-state buffers, pass transistors) on or off. It is fast and fully reconfigurable, at the cost of being volatile. This is the same idea behind the configuration SRAM inside every FPGA we meet later in this book.

UV-erasable memory stores its configuration differently. Its key element is a MOSFET with a second gate, electrically isolated and sandwiched between the control gate and the substrate, called a floating gate. A strong lateral field between drain and source injects electrons onto that floating gate, and because it has no conductive path to leak through,

those electrons stay trapped once the field is removed, even with power off. Erasing the device means shining UV light on it, energetic enough to let the trapped electrons escape.

Electrically erasable memory keeps the same floating-gate mechanism, but the device is built so the trapped charge can be removed electrically instead, without needing a UV lamp.

Fuse and antifuse technologies work differently again, and only in one direction. A fuse is a narrow bridge of conducting material that blows open when a programming current is forced through it. An antifuse is the opposite: a thin dielectric separating two conductive layers, which ruptures under a programming voltage and creates a permanent short between them. Either way, programming a fuse-based device is a one-time, permanent decision, unlike the other three technologies, which can all be reconfigured.

2.4.2 Simple programmable logic devices

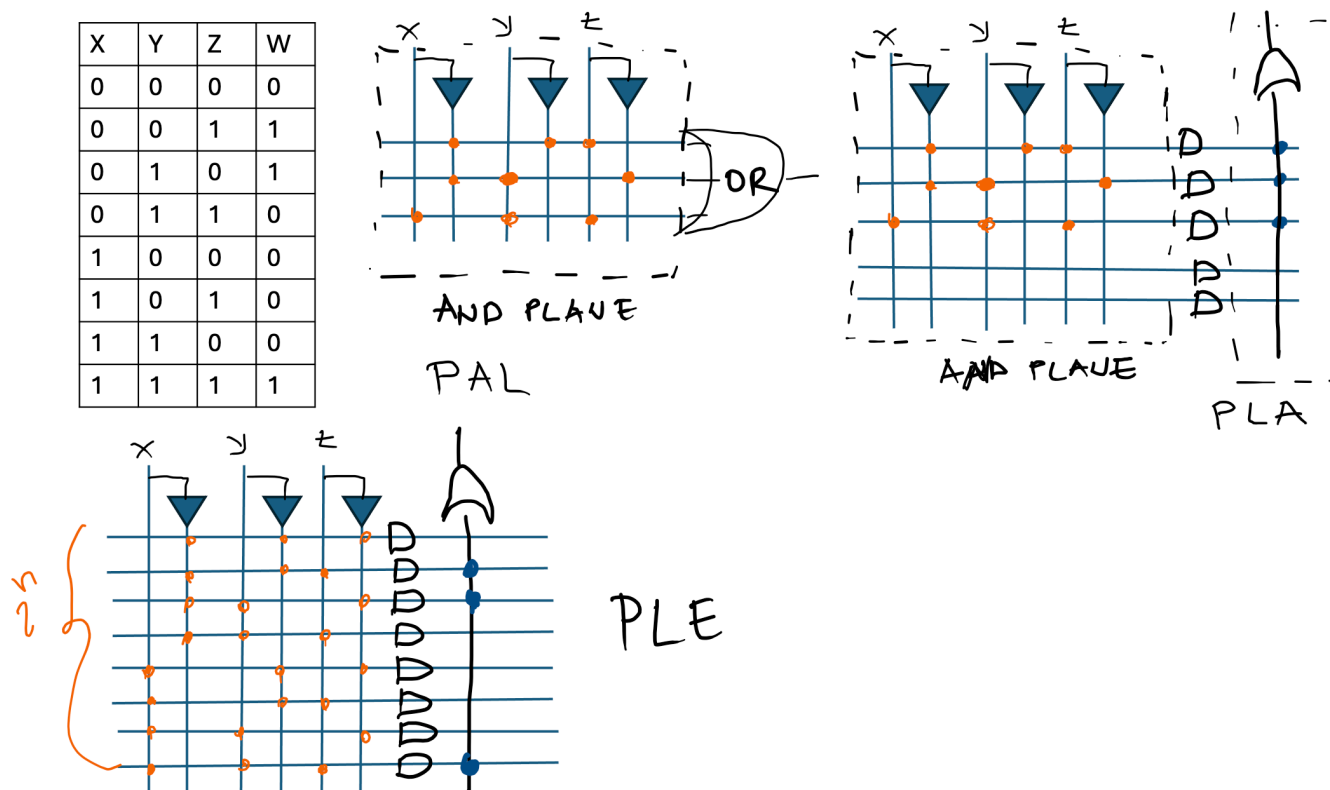


Figure 2.2: SPLD architectures: PAL (configurable AND plane, fixed OR plane), PLA (both planes configurable), and PLE (fixed AND plane covering every product term, configurable OR plane)

Simple programmable logic devices (SPLDs) are the direct ancestors of the FPGA, and the simplest configurable logic devices we will meet. Internally, an SPLD is built from one or two programmable logic planes, an AND plane and an OR plane, that implement a digital function exactly the way we would write it out by hand: as a sum of products. Every input, and its inverted value, runs as a vertical line through the AND plane, and every horizontal line represents one product term. Which vertical lines feed which horizontal line, and which product terms feed which output, is exactly what gets configured.

Which of the two planes is configurable, and which is fixed, gives us three SPLD variants. In a PAL (Programmable Array Logic) device, the AND plane is configurable but the OR plane is fixed. In a PLA (Programmable Logic Array), both planes are configurable, at the cost of more configuration bits and a larger device. In a PLE (Programmable Logic Element), the AND plane is fixed and already generates every possible product term, all 2^n of them for n inputs, and only the OR plane, choosing which of those terms feed each output, is configurable.

2.4.3 Complex programmable logic devices

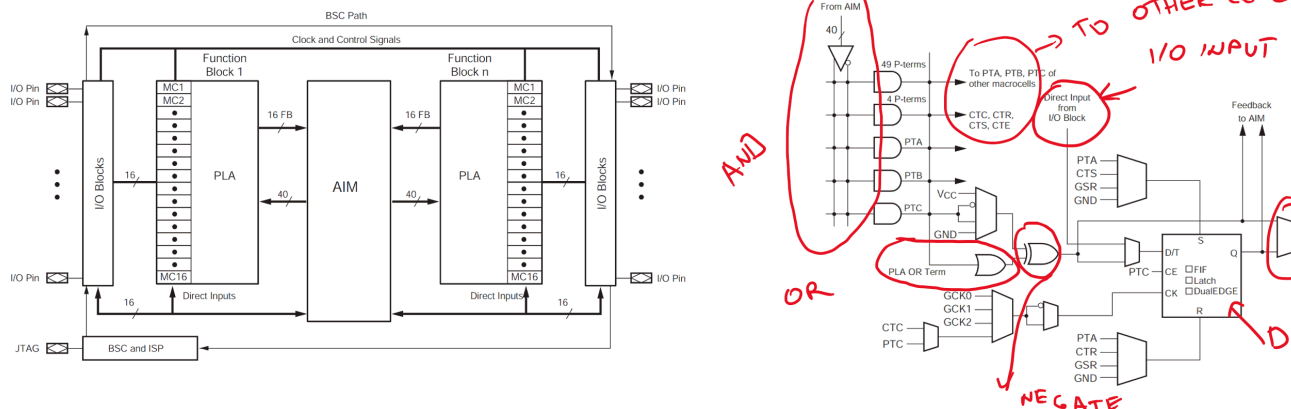


Figure 2.3: CoolRunner-II CPLD architecture: I/O blocks and function blocks tied together by the Advanced Interconnect Matrix (AIM), with one function block's macrocell detail (AND/OR terms, an XOR stage, and a configurable flip-flop) on the right (source: adapted from Xilinx CoolRunner-II documentation, DS090)

A complex programmable logic device (CPLD) is, at its core, many SPLDs packaged on one chip and tied together with a programmable interconnect, trading a single SPLD's limited

capacity for a much larger, still fairly predictable, device.

Take the Xilinx CoolRunner-II as a concrete example. Its I/O blocks connect the chip to the outside world, the same role they play in the FPGAs we meet later in this book. Thirty-two identical function blocks hold the actual logic, and an Advanced Interconnect Matrix (AIM) wires the function blocks to each other and to the I/O blocks, playing the same role the FPGA's routing plane plays later.

Each function block looks like a small PAL: 56 AND product terms feeding 16 OR sum outputs. Beyond that core, each output can pass through an XOR stage, useful for controlled inversion or parity-style logic, and into a flip-flop resource configurable as a D flip-flop, a level-sensitive latch, or a toggle (T) flip-flop. An output mux then picks whether the macrocell's output is the raw combinational result or the registered one, the same combinational-versus-registered choice the FPGA's CLB output mux makes.

Tiling many PAL-like blocks together through the AIM lets a CPLD implement far larger combinational and sequential designs than a single SPLD could, with timing that stays more predictable, and I/O that responds faster, than the larger and more routing-heavy FPGAs we look at later in this book.

3 Introduction and Review of Fundamentals

Introduction

(placeholder — fill in with week 1 content: Si -> CMOS -> digital circuits -> FPGA, a fast recap)

4 FPGA Fundamentals

FPGA Fundamentals

4.1 Goal for this week

- Place the FPGA on the spectrum between a general-purpose CPU and a fixed ASIC, and see why that position makes it the right prototyping target for this course
- Break the FPGA fabric down into its three building blocks: CLB, routing plane, I/O blocks
- Open up a CLB far enough to see the LUT + flip-flop pair that will later become “the thing our synthesized SystemVerilog turns into”

4.2 Why reconfigurable hardware?

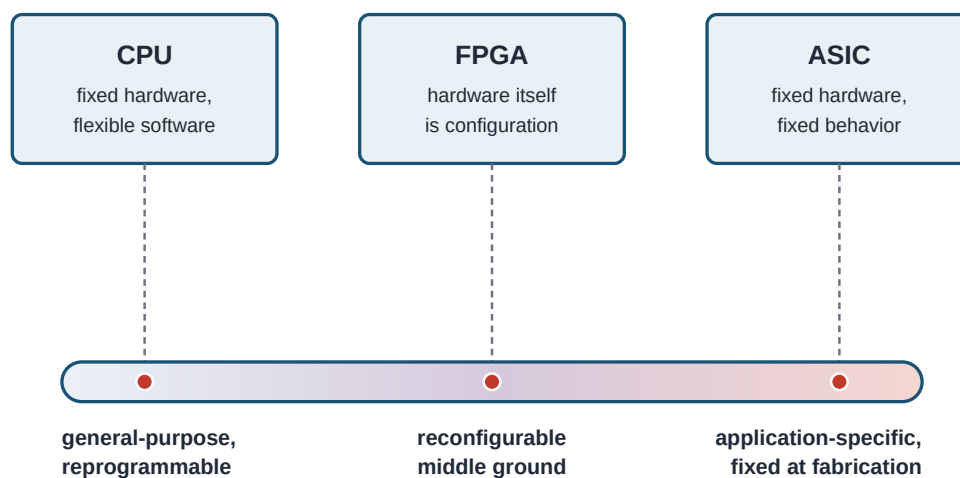


Figure 4.1: The CPU-FPGA-ASIC spectrum: flexibility and reprogrammability trade off against efficiency and fixed-function performance

As we saw in previous chapters, a CPU is a general-purpose integrated circuit: its function is defined not by its hardware but by the software program a developer writes for it. Development cost and time are low, since a single instruction set can be programmed in general-purpose or domain-specific languages to meet almost any demand — from database search to signal processing and beyond. But this same generality is also the CPU’s biggest disadvantage. A CPU performs well on workloads like databases, yet it can be slow on compute-intensive problems such as neural network inference.

ASIC designs overcome this general-purpose bottleneck by mapping the application directly into hardware rather than software. A well-known example is Google’s Tensor Processing Unit (TPU), built to accelerate the compute-intensive workloads of neural network inference. Its designers tailored the architecture to that one problem, wiring together a matrix of arithmetic logic units (ALUs) instead of following the standard CPU approach of a five-stage pipeline. Comparing a CPU and a TPU on neural network inference, the TPU wins decisively — but hand it a database search instead, and it is useless. This is the core limitation of ASIC accelerators: each one targets a single application, and once fabricated, it cannot be repurposed for anything else.

Comparing the CPU and the TPU, what we would really like is both properties at once: the flexibility to repurpose the device, and hardware tailored to the application at hand to squeeze out its full potential. Reconfigurable hardware, the field-programmable gate array (FPGA), gives us both, and its name spells out how. “Gate array” comes from the internal structure: logic blocks arranged on a matrix-like fabric, echoing the gate-array architecture used in some ASICs. “Programmable” does not mean the FPGA executes a software program in the traditional sense, though it can in a limited way. Instead, it refers to the reconfigurable property itself, achieved by loading a bitstream, a special binary configuration file, onto the device. “Field” refers to the field of application: an FPGA can adapt its internal structure to whatever application is at hand, out in the field, long after manufacturing.

This is exactly the trade-off an FPGA offers. It sits between the CPU and the ASIC by turning the hardware itself into a form of software: the logical function of every building block, and the wiring between them, is set by the bitstream loaded after manufacturing. We get most of an ASIC’s advantage of “hardware built for the job” without paying its fixed non-recurring engineering (NRE) cost or its all-or-nothing commitment. That is exactly why this course targets FPGAs. A mistake in a lab assignment costs a re-synthesis and a few minutes, not a re-spun die. FPGAs are also the standard vehicle for rapid prototyping and hardware acceleration in industry for the same reason. Reconfigurability turns hardware design into something you can iterate on.

4.3 The three building blocks

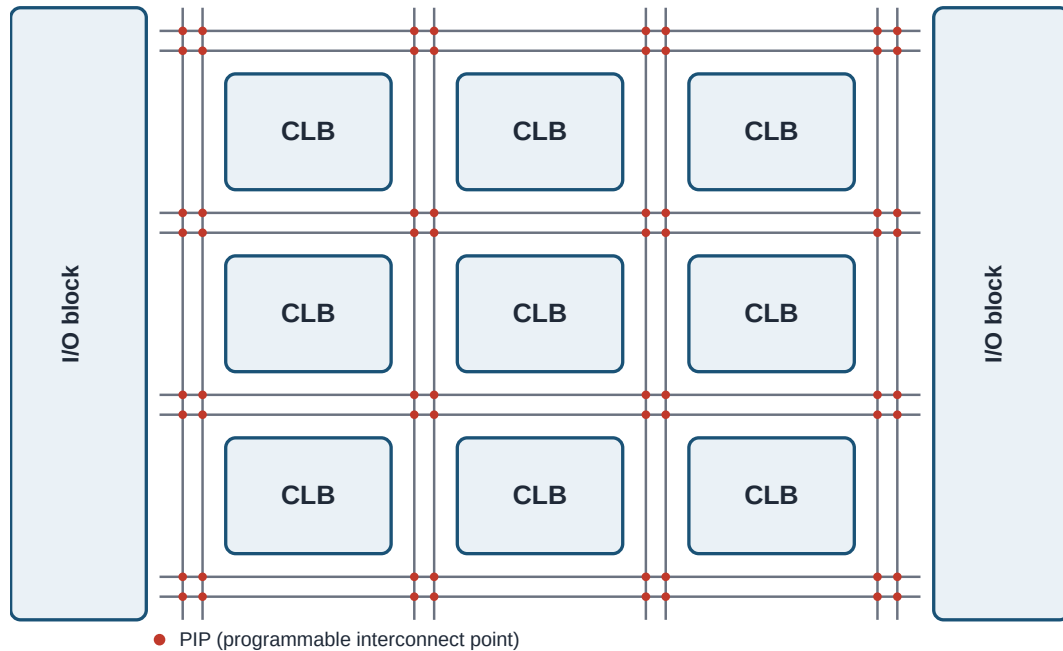


Figure 4.2: FPGA fabric: CLBs embedded in a routing plane, flanked by I/O blocks

Every FPGA, regardless of vendor or generation, is built from the same three kinds of block. CLBs (also called logic elements) sit in a regular grid and implement the actual logic functions and storage of the design. Between and around them runs the routing plane: a fabric of wires and programmable switches whose only job is to connect a CLB's outputs to the inputs of other CLBs, in whatever pattern the design needs. At the edges of the die, I/O blocks connect the fabric to physical pins, translating between the FPGA's internal signal levels and whatever the outside world expects. None of these three blocks is useful without the other two: a CLB with no routing to reach it is dead logic, and routing with nothing to connect is just wire. Placing a design onto an FPGA is precisely the problem of deciding which CLB implements which piece of logic (placement) and which routing resources carry the signals between them (routing). We will meet both steps again when we look at the FPGA design flow.

4.3.1 Configurable Logic Block (CLB)

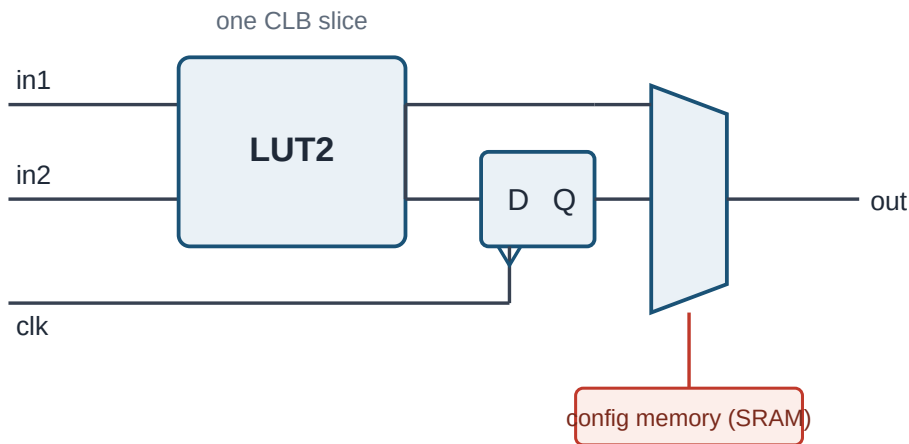


Figure 4.3: One CLB slice: a LUT2 and a flip-flop, with the output mux picking between the combinational and registered result

The CLB is the main carrier of reconfigurability in an FPGA and its elementary building block. You can think of the CLB as the FPGA equivalent of an assembler instruction, the most basic element everything else is built from. The term itself differs across vendors. Xilinx calls it a logic slice, while Altera calls it a logic element. Either way, they refer to the same thing. As the primary building block for digital circuits, a CLB must support combinational logic, sequential logic, and combinations of the two. It consists of a look-up table (LUT), a flip-flop, and an output mux.

A LUT implements combinational logic through memory lookup rather than by wiring gates together. Structurally, it consists of SRAM (static random-access memory) cells paired with a multiplexer that selects which cell's content is forwarded to the output, based on the signal inputs. To see this concretely, consider the smallest useful case, a two-input LUT, or LUT2. With two inputs there are four possible input combinations, so a LUT2 is built from four memory cells and a 4-to-1 multiplexer. The two inputs act as the multiplexer's select lines, in other words as an *address*, and the output is simply whatever bit is stored in the addressed cell.

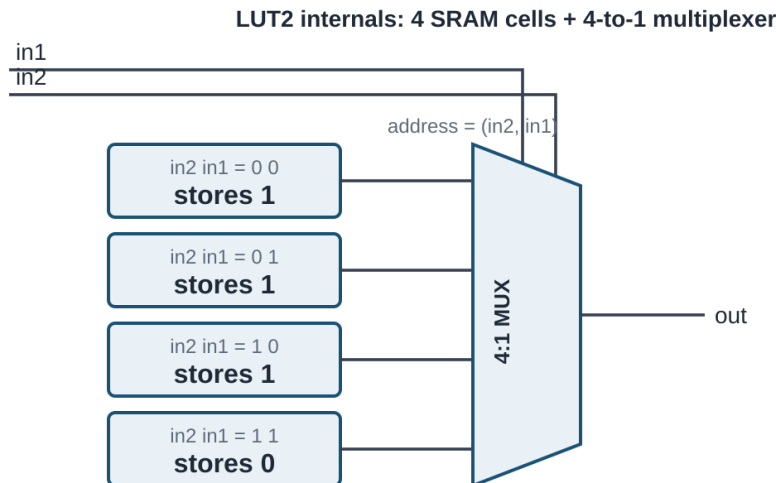


Figure 4.4: LUT2 internals: the two inputs address one of four SRAM cells, and the addressed cell’s stored bit is forwarded to the output

To implement logical functions with two input variables, you need to map the truth table into SRAM cells. For example to implement a two-input NAND on a LUT2, we do not build a NAND gate at all: we store the NAND truth table’s output column (1, 1, 1, 0) into the four memory cells, in the order addressed by the two inputs, and the LUT now behaves as a NAND for as long as that configuration is loaded. The reconfigurability of LUTs is what makes FPGAs so flexible: the same LUT can just as easily hold an AND, an XOR, or any other two-input function. Reconfiguring the FPGA never touches a wire between gates. It only rewrites these memory cells. Because those cells are ordinary SRAM, they are volatile: power off the board and the “circuit” is gone, which is why every FPGA design needs a configuration step after every power-up.

We do not build larger LUTs as one giant memory array wired to a huge multiplexer. Instead, real FPGA architectures build them recursively from smaller LUTs, adding one extra input and one extra multiplexer stage at a time. A LUT3 is nothing more than two LUT2s, both fed by the same two inputs, with their outputs chosen between by a 2:1 mux controlled by the third input. Extending the same idea, a LUT4 is built from two LUT3s, each in turn built from two LUT2s, combined by one more 2:1 mux controlled by the fourth input. Each additional input doubles the number of underlying LUT2 building blocks and adds one more layer of multiplexing on top. This recursive structure is exactly why vendors describe their larger LUTs as fracturable: the same physical hardware that implements one LUT4 can, with a small configuration change, split back into two independent LUT3s, or four independent LUT2s, letting the synthesis tool pack more, smaller functions into fewer CLBs when a design does not need the full input width.

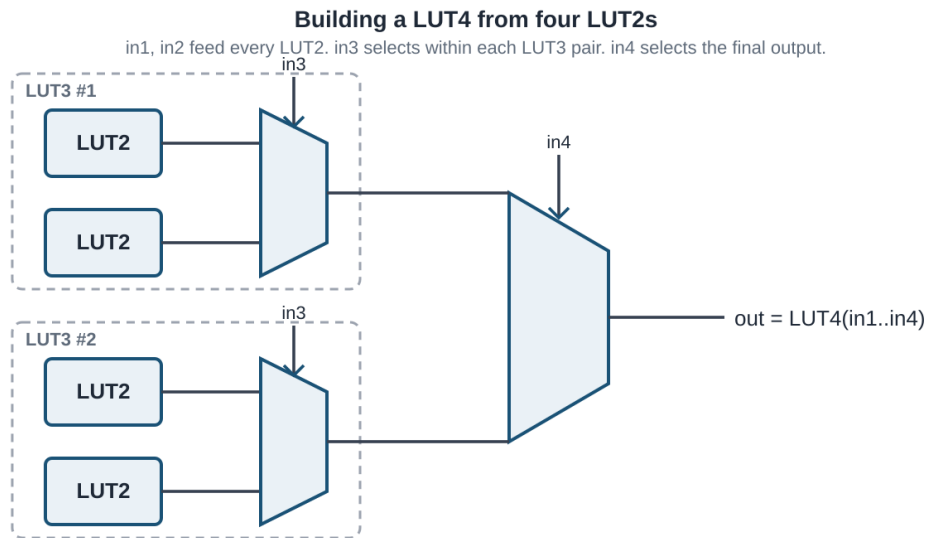


Figure 4.5: Building a LUT4 from four LUT2s: two levels of 2:1 muxing, selected by in3 and in4

Few real digital circuits are purely combinational. Almost every design also needs elements that are synchronous to a clock. To support that, every CLB slice also includes a D flip-flop (DFF) fed from the LUT's output. As we saw earlier, a DFF copies its input (D) to its output (Q) on the active edge of the clock signal, *clk*. Through configuration bits, the developer can adapt that DFF's behavior, choosing whether it triggers on the rising or falling edge, and setting its initial or reset value. This exposes yet another layer of reconfigurability inside the CLB.

The last element of the CLB is the output mux, which picks between the raw LUT output and the flip-flop's registered output. That mux's select line is, like the LUT's memory cells, just another bit in configuration SRAM. The same physical slice can act as combinational logic or as a single-bit register, purely depending on how it is configured. Real devices bundle several LUT + flip-flop slices, wider LUTs, and extra logic (carry chains, muxes for combining LUTs into wider functions) into one CLB. We look at a concrete example of that when we cover a real device architecture later.

4.3.2 Routing plane: programmable interconnect

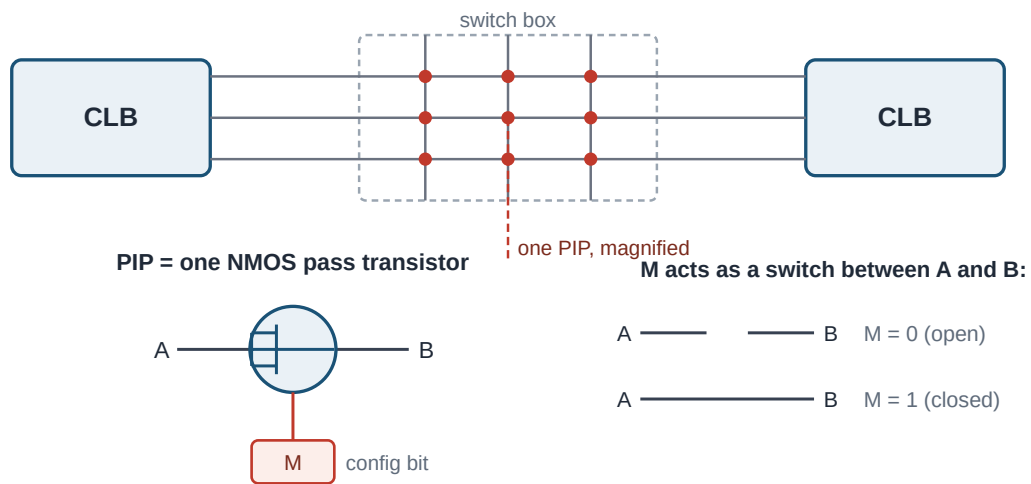


Figure 4.6: Two CLBs connected through a switch box. A PIP is one config-bit-controlled NMOS pass transistor

Routing plays two closely related roles. First, it lets many individual CLBs act as one large digital design: a real circuit needs far more logic and storage than a single CLB slice can hold, so the design is spread across hundreds or thousands of CLBs, and routing is what stitches their individual outputs together into one coherent datapath. Second, routing connects that internal logic to the outside world, joining CLBs to the I/O blocks that carry signals in and out of the fabric. Without it, CLBs and I/O blocks would be correct but disconnected islands of logic.

Between the rows and columns of CLBs run wiring tracks, and at every point where a horizontal and a vertical track cross, a switch box can optionally join them. The atomic component of a switch box is the PIP: a single NMOS transistor used purely as a switch, with its gate driven by one configuration SRAM bit. When that bit is 0, the transistor is off and the two tracks it sits between are electrically separate. When it is 1, the transistor conducts and the tracks become one net. A design's entire wiring, meaning every connection from one CLB's output to another CLB's input, reduces to which of these thousands of PIPs are turned on. This is also why the router is typically the slowest step in the FPGA implementation tools: placement decides which CLB does what, but routing has to find, among a huge and only partially connected mesh of tracks and switch boxes, one specific set of PIPs that realizes every net in the design without any two nets shorting together.

4.3.3 I/O blocks

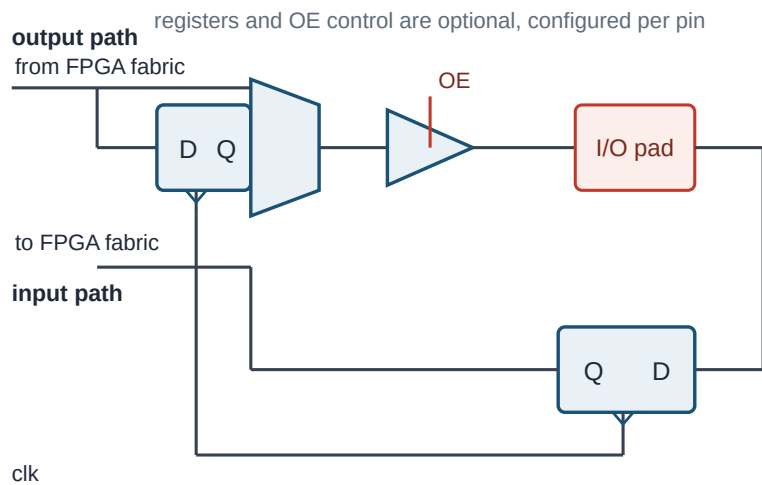


Figure 4.7: I/O block output and input paths: an optional register and a tri-state buffer on output, an optional register on input

An I/O block is the boundary between the reconfigurable fabric and a fixed physical pin, and it has to support more than “just wire a signal through.” On the output side, a signal from the fabric can optionally be registered right at the pin before it reaches a tri-state buffer. That buffer’s output-enable (OE) input decides whether the pad is actively driven at all, which is what makes a genuinely bidirectional pin possible. The same pad can be released to high-impedance and read as an input whenever the design is not driving it. The input side mirrors this with its own optional register. Registering directly in the I/O block, rather than in the nearest CLB, shortens the path the signal has to travel before it is captured, which matters because I/O timing margins are usually tighter than internal fabric timing. As with everything else on the chip, whether each of these registers is used, and whether a pin behaves as input, output, or bidirectional, is a configuration decision, not a fact fixed at fabrication.

4.3.4 Reconfigurability, level by level

We have now seen five separate places where a single configuration SRAM bit changes what the FPGA does. Inside the CLB, one set of bits picks the function stored in each LUT, another set picks whether the D flip-flop triggers on the rising or falling edge and what value it resets to, and one more bit decides whether the slice’s output is the LUT’s raw combinational result or the flip-flop’s registered one. Out in the routing plane, a bit per PIP decides which pairs of tracks are joined into the same net. At the I/O block, bits decide whether a register is

used on the input or output path, and whether a pin drives, receives, or does both. None of these is a special case. Every one of them is the same primitive: an SRAM cell whose value the configuration step writes before the device does anything useful. What looks like five different kinds of reconfigurability is really one mechanism, repeated at every point in the fabric where a design decision has to be made. This is also, in one sentence, what a bitstream is: the full list of values for every one of these bits, for every CLB, PIP, and I/O block on the device.

4.4 Case study: Xilinx Series-7

We have described the CLB, the routing plane, and the I/O block as three abstract building blocks, present in every FPGA regardless of vendor or generation. To see how a real device turns those abstractions into a concrete architecture, we now look at a specific family: Xilinx Series-7. Series-7 comprises four sub-families, Spartan-7, Artix-7, Kintex-7, and Virtex-7, all built from the same underlying fabric while trading off cost, logic capacity, and signal-processing performance. Spartan-7 targets the low end of that range: small, cost-sensitive, high-volume applications. Virtex-7 sits at the opposite end, aimed at the highest connectivity bandwidth, logic capacity, and DSP throughput. Artix-7 and Kintex-7 fill the middle ground. Every member of the family, from the smallest Spartan-7 to the largest Virtex-7, is assembled from the same handful of building blocks described below; only their quantity and mix change from device to device.

4.4.1 CLB: SLICEL and SLICEM

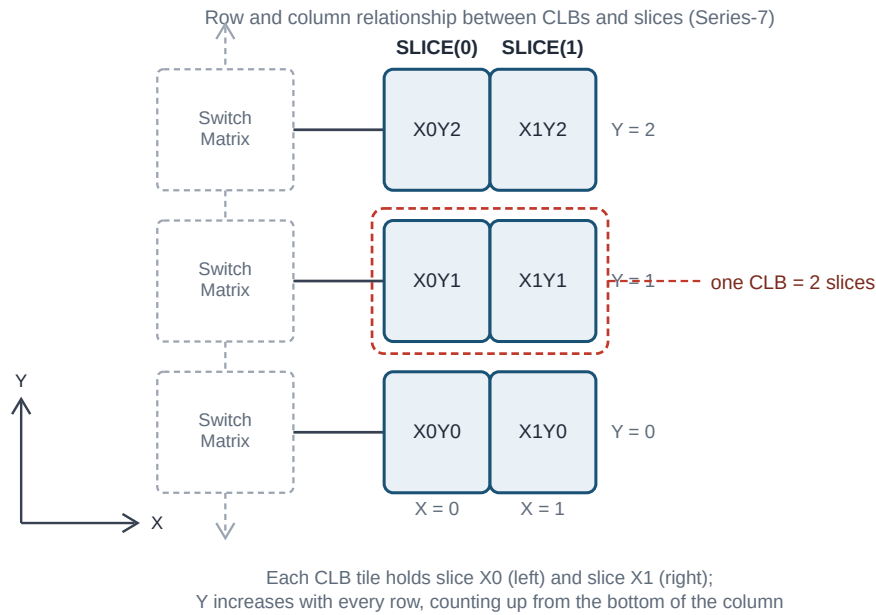


Figure 4.8: Row and column relationship between CLBs and slices: each CLB tile holds slice X0 and slice X1, wired to its switch matrix, with Y counting up from the bottom of the column (redrawn from AMD/Xilinx UG474, Fig. 1)

Where the generic CLB we described earlier held one LUT and one flip-flop, a Series-7 CLB is considerably richer, and Xilinx builds it from two smaller units called slices. Two slice types exist. Every CLB is either a CLB0, containing two SLICEL (“slice logical”) slices, or a CLB1, containing one SLICEM (“slice memory”) and one SLICEL slice. Each slice, of either type, contains four six-input LUTs (LUT6) rather than the single LUT2 we used to introduce the idea — the same recursive fracturable-LUT construction we saw earlier, just taken further, and each LUT6 can equally well operate as two independent LUT5s sharing the same five inputs. To build functions wider than six inputs, the slice adds dedicated combining multiplexers on top of the LUTs themselves: F7AMUX and F7BMUX each merge a pair of LUT6 outputs into one seven-input function, and a further F8MUX merges the two resulting LUT7 outputs into a single eight-input function. This is the same output-mux idea from the generic CLB, just repurposed to widen LUTs instead of choosing between a combinational and a registered output.

Because addition is one of the most common operations in digital design, each slice also includes a dedicated carry chain: fast, purpose-built hardware that propagates carry signals

between adjacent slices without routing them back through the general interconnect, considerably speeding up adders and comparators compared to building the same carry logic out of ordinary LUTs. On the sequential side, each slice packs eight flip-flops rather than the one flip-flop per LUT we introduced earlier. Four of these are configurable to behave as either an edge-triggered flip-flop or a level-sensitive latch, while the remaining four are flip-flop only; all eight share one clock and one clock-enable signal, though each still has its own configuration bits for init value and set/reset behavior, echoing the per-flip-flop re-configurability we discussed for the generic CLB.

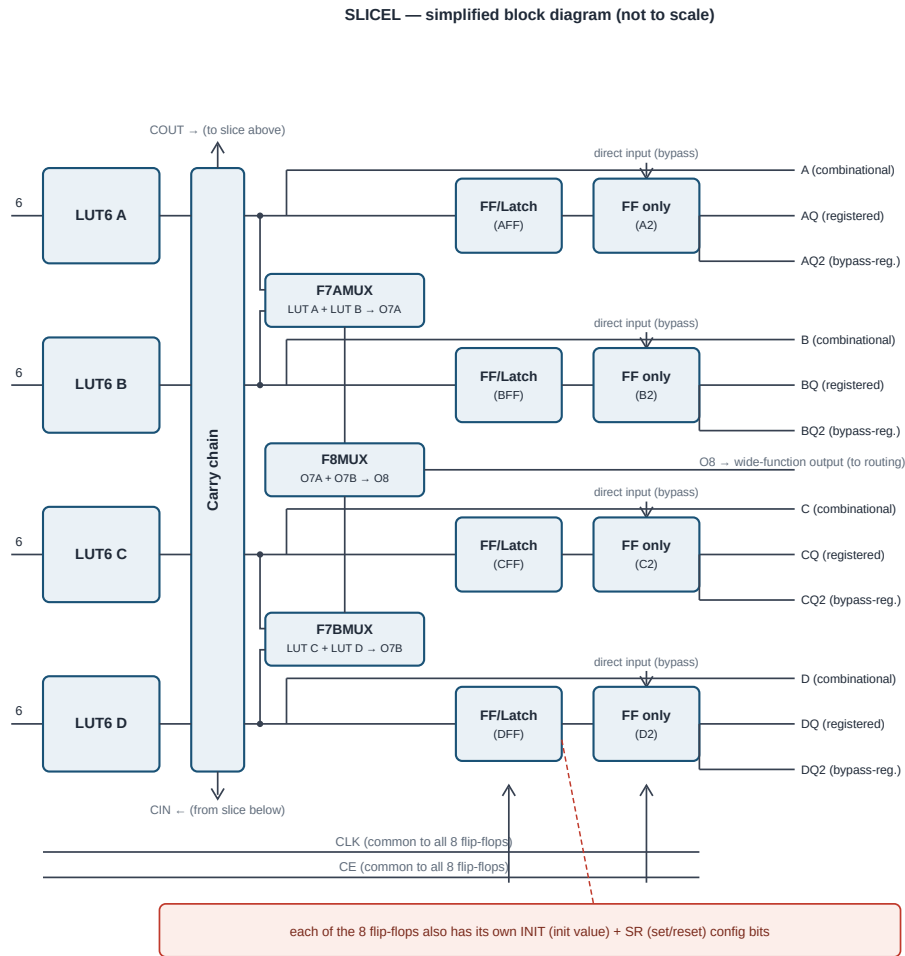


Figure 4.9: SLICEL, simplified block diagram: four LUT6s feed the F7A/F7B/F8 combining muxes and the carry chain, each lane ending in an FF/Latch and an FF-only register sharing one clock and clock-enable (redrawn, not to scale, after AMD/Xilinx UG474, Fig. 2)

SLICEM slices add one more capability that SLICEL slices lack. A LUT's whole purpose, as we saw earlier, is to hold a fixed truth table in SRAM cells that the design's inputs address; that content does not change while the design runs. SLICEM's LUTs can instead be repurposed as small, fast, genuinely writable memory: either distributed RAM, where the LUT's SRAM cells are written and read directly by the design as if they were a tiny local memory, or a 32-bit shift register, where the same cells behave as a chain of delay elements. Both uses trade away the LUT's normal role as a combinational lookup table in exchange for a small amount of local storage exactly where it is needed, without consuming a dedicated block RAM.

4.4.2 Block RAM and DSP slices

SLICEM's distributed RAM is useful for small amounts of storage, but real designs routinely need much larger memories: register files, FIFOs, line buffers, caches. Building those out of LUTs alone would be both wasteful of logic resources and slow, so Series-7 devices additionally provide dedicated block RAM (BRAM): hard-wired memory macros scattered through the fabric, entirely separate from the CLBs. Each BRAM comes in one of two hard-wired sizes, 36 Kb or 18 Kb, and each size can be configured, at implementation time, into a range of width and depth combinations — a 36 Kb block, for instance, can be organized as 32K words of 1 bit, 16K words of 2 bits, 8K words of 4 bits, and so on, trading width for depth as the design requires. Configuration also selects the memory's port structure: single-port, true dual-port (two fully independent address, read, and write ports), or simple dual-port (one port dedicated to writing, the other to reading). A FIFO is not a separate hardware primitive at all; it is simply a block RAM combined with dedicated pointer and flag logic that turns its linear address space into a first-in-first-out queue.

Arithmetic is common enough, and multiplication expensive enough to build efficiently out of general LUT fabric, that Series-7 devices dedicate a third kind of hard-wired block to it: the DSP slice. A DSP slice implements a fixed pipeline of multiply, add, and accumulate hardware directly in silicon, entirely bypassing the LUT and carry-chain logic used for general-purpose addition. Just as with the CLB and BRAM, a DSP slice is configured rather than wired by hand — configuration bits select which of its internal operations are active and how they are chained together — but the arithmetic itself runs on dedicated hardware rather than being synthesized from LUTs, which is what makes DSP slices dramatically faster and more area-efficient than an equivalent LUT-based multiplier.

4.4.3 Clock management and interconnect

A clock signal is unlike any other net on the chip: every register that uses it must see the same edge at very nearly the same instant, or the design's timing margins erode. General-purpose

routing, built from PIPs and switch boxes with no particular guarantee on delay matching, is not good enough for this job, so Series-7 devices route clocks through a dedicated network instead. Global clock buffers distribute a small number of clock signals across the device with deliberately low skew, and the device's area is partitioned into clock regions, each spanning 50 CLB rows, with its own local clock distribution so that skew stays low even far from the clock's entry point. Before a clock reaches those buffers, it can optionally pass through a clock management tile (CMT), dedicated hardware that synthesizes new frequencies, shifts phase, or removes accumulated skew — capabilities well beyond anything the general fabric could provide.

Everything that is not a dedicated clock net still relies on the same general-purpose interconnect introduced earlier: horizontal and vertical routing tracks running between rows and columns of CLBs, joined at switch boxes wherever they cross. A switch box is simply a cluster of PIPs, and, exactly as in the generic architecture, each PIP reduces to one NMOS pass transistor gated by a single configuration SRAM bit. Series-7 does not introduce a new mechanism here at all — it is the same one-bit-per-connection primitive met earlier, just deployed at the scale a real device needs: thousands of switch boxes, each with many PIPs, giving the router enough freedom to connect any CLB, BRAM, or DSP slice's output to any other block's input.

4.5 FPGA design flow

Earlier, when introducing routing, we noted that placing a design onto an FPGA comes down to two questions: which CLB implements which piece of logic, and which routing resources carry the signals between them. The FPGA design flow is the sequence of tool steps that answers those questions automatically, turning an RTL description into the bitstream that configures every SRAM bit we have discussed in this chapter. That flow closely resembles the one used for ASICs, since both ultimately map a hardware description down to physical resources on silicon, but it differs in one important respect: the target primitive throughout is a LUT and its associated flip-flop, not a standard cell from an ASIC library, which shapes what several of the intermediate steps actually do.

The flow begins with design entry, typically RTL written in a hardware description language, and proceeds to synthesis. Synthesis interprets that RTL and produces a synthesized circuit netlist, generally stored in the standard EDIF format, along with preliminary performance estimates that give an early, optimistic sense of whether the design will meet its timing goals before the slower steps that follow. A translate step then merges this netlist with any others belonging to the same design (for instance, netlists from separately synthesized IP blocks) into one single netlist: a text description of every component in the design and how its terminals connect to every other component's terminals.

The next two steps are where the LUT-based nature of FPGAs really shows. Mapping takes the logical symbols in that merged netlist, ordinary gates such as ANDs, ORs, and multiplexers, and groups them into the physical components the target device actually provides: LUTs of a specific width, flip-flops, carry-chain segments, and so on. This is conceptually similar to technology mapping in an ASIC flow, except the “cell library” being mapped to is not a fixed set of gates but the fracturable-LUT building blocks we described earlier in this chapter. Placing then takes every one of those mapped components and assigns it to one specific site on the chip, a particular slice within a particular CLB, deciding, in other words, which CLB implements which piece of logic. Routing follows, connecting every placed component to every other one it needs to reach by selecting a path through the switch boxes and PIPs of the routing plane, deciding which routing resources carry each signal.

Routing also produces the reports that tell us whether the design actually works at the clock frequency we want. A static timing analyzer performs a static analysis of the fully placed and routed circuit, accounting for every source of delay along every path, logic delay through LUTs, routing delay through PIPs, and everything in between, to identify the circuit’s critical path: the longest path from the output of one register to the input of another. Because a synchronous design can run no faster than its slowest register-to-register path allows, the critical path directly determines the design’s maximum clock frequency, which is exactly the number this whole flow exists to maximize.

Once placement and routing succeed and timing closes, the last step is bitstream generation: writing out the configuration file for the FPGA, the complete list of values for every LUT content bit, every flip-flop’s configuration, every PIP, and every I/O block setting that we introduced across this chapter. That bit file can be downloaded directly onto the FPGA, configuring its (volatile) SRAM cells as we described earlier, or converted into a PROM file: a non-volatile store that reprograms the FPGA automatically, from power-up, every time the board is powered on.

5 SystemVerilog — Syntax and Synthesizable Coding Style

SystemVerilog basics

5.1 Goal for this week

- Learn to read SystemVerilog by knowing exactly where the C-like syntax stops being a reliable guide
- Separate combinational logic (`always_comb`) from sequential logic (`always_ff`), and the assignment operator each one requires
- Recognize latch inference and reset styles on sight, before they show up as bugs in your own pipeline stages

5.2 It looks like C. It isn't.

The snippet above is syntactically close enough to C that it is tempting to read it top to bottom, one statement after another. That reading is wrong, and it is the single most important thing to unlearn this week. A module is not a function that runs once and returns — it is a piece of hardware that exists for as long as the circuit is powered. `y1` and `y2` are not computed “first” and “second”; they are two separate wires, each continuously driven by its own gate, both existing simultaneously and forever. There is no call stack, no return value, and no “after” — only signals that settle to a value whenever their inputs change.

Every construct in this chapter is a variation on this same fact: SV syntax borrows C's punctuation, but describes structure and behavior in time, not a sequence of steps.

5.3 Signals vs. variables

A C variable holds a value because you assigned it one; nothing drives it otherwise. A logic signal in SV is closer to a wire with memory — it holds whatever it was last driven to, by whichever `assign` or `always` block is responsible for it, and it is an error (caught by simulation, not always by synthesis) for more than one such block to drive it.

```
typedef enum logic [1:0] {IDLE, RUN, DONE} state_t;

typedef struct packed {
    logic    valid;
    logic [7:0] data;
} packet_t;
```

`typedef enum` gives a set of named constants a single type, instead of scattering magic numbers through the code. `typedef struct packed` bundles related signals into one name that can travel through a module’s ports or a pipeline register as a single unit — you will see exactly this pattern in `pipeline_pkg.sv` next chapter, where control signals are grouped this way instead of being passed as individual wires.

5.4 Combinational logic: `always_comb`

An `always_comb` block re-evaluates itself immediately whenever any signal it reads changes — closer to a spreadsheet formula that recalculates on every input edit than to a function you call. Assignment inside it uses `=` (blocking), read as “this is what the output equals right now,” not “do this step, then the next.”

The example above is an incomplete 2-to-1-style decoder: two of the four `sel` cases are unhandled. In a spreadsheet, an unhandled case would just show a blank cell. In hardware, `y` cannot go blank — if no branch assigns it, the only way to keep `y` at *some* value is to remember the previous one. Synthesis does exactly that: it infers a latch. This is almost never what you want in a synchronous design, and it is the most common bug `always_comb` produces. The fix is always the same: every signal assigned in an `always_comb` block must be assigned on *every* path through it — in practice, an `else` or a `default` branch on every `if/case`.

5.5 Sequential logic: `always_ff`

`always_ff` describes a block that updates only at a clock edge — a register, not a formula. It uses `<=` (non-blocking assignment), which reads as “at this clock edge, schedule this update,” rather than `=`’s “this is the value right now.” The distinction matters the moment two `always_ff` blocks depend on each other, which is exactly the situation every pipeline register is in.

Consider two registers meant to swap values on the same clock edge:

```
always_ff @(posedge clk) begin
    a <= b;
    b <= a;
end
```

With non-blocking assignment, both right-hand sides are read using each register’s value *before* this clock edge, and both updates take effect together — the swap works, matching what “on every clock edge, in parallel” should mean. Replace `<=` with `=` and the second line reads the value the first line just wrote, collapsing two parallel updates into a sequential one — the swap silently fails, and nothing about the syntax warns you. This is why the pairing on the previous slide has no exceptions: `=` in combinational logic, `<=` in clocked logic, every time.

5.6 Reset styles

Both styles produce a register that clears `q` to zero on reset; they differ in *when* the clear can happen. A synchronous reset only takes effect at a clock edge — `rst` is just another input to the same logic that decides the next value of `q`. An asynchronous reset takes effect the moment `rst` asserts, independent of the clock, which is why it appears in the sensitivity list alongside `posedge clk`.

The DFF you will build next chapter as a pipeline register uses exactly this pattern — it is worth being able to tell the two styles apart on sight, since the choice affects how quickly a stalled or flushed pipeline stage can be forced back to a known state.

5.7 From syntax to the pipeline

Every construct introduced in this chapter reappears immediately in the next one. `typedef enum` and `typedef struct packed` are how `pipeline_pkg.sv` groups control signals; `always_ff` with the non-blocking assignment and reset style you just saw is how `if_id_reg` and the other pipeline registers are built; `always_comb` with the blocking assignment and complete-branch discipline is how the ALU and decoder avoid the latch-inference trap. Nothing in the pipeline chapter introduces new syntax — it only puts this chapter’s syntax to work on a larger example.

6 Five-Stage Pipeline — Structure

Five-stage pipeline

6.1 Goal for this week

- Translate the well-known pipeline theory (IF/ID/EX/MEM/WB) into SystemVerilog
- Apply SystemVerilog syntax immediately on a real, motivating example
- Treat memory (for now) as an interface — the implementation follows next week

We implement pipeline registers as separate modules for two reasons. First, pedagogically: each such register is a direct, visible example of a D flip-flop (DFF) in the context of a real processor, which was an explicit goal of the second part of the course. Second, practically: separate modules with a fixed interface (see `pipeline_pkg.sv`) allow us to substitute an individual stage with a reference (“golden”) implementation in the jigsaw testing methodology — discussed in more detail in the verification chapter. This five-stage IF/ID/EX/MEM/WB structure follows the classic pipelined datapath described by Patterson and Hennessy (2017).

6.2 Modules for this week

- `if_stage`, `if_id_reg`
- `id_stage` (`regfile`, `imm_gen`, `decoder`)
- `id_ex_reg`
- `ex_stage` (`alu`)
- (`mem_stage`, `wb_stage` — interface only for now, implementation next week)

Placeholder — replace with the exported pipeline module diagram

Figure 6.1: Modular pipeline structure

6.3 Example: pipeline_pkg.sv

```
package pipeline_pkg;

    typedef enum logic [3:0] {
        ALU_ADD, ALU_SUB, ALU_AND, ALU_OR, ALU_XOR,
        ALU_SLL, ALU_SRL, ALU_SRA, ALU_SLT, ALU_SLTU
    } alu_ctrl_t;

    typedef struct packed {
        logic mem_read;
        logic mem_write;
        logic [1:0] mem_size;
        logic mem_unsigned;
    } mem_ctrl_t;

    typedef struct packed {
        logic reg_write;
        logic [1:0] wb_sel;
    } wb_ctrl_t;

endpackage
```

6.3.1 Why a struct instead of individual signals

`typedef struct packed` is standard synthesizable SystemVerilog (IEEE 2017), so this pattern is portable across the tools you will use this semester.

An alternative to the approach above would be for each control signal (`mem_read`, `mem_write`, `reg_write`, ...) to travel through the pipeline as a separate wire and a separate field in every pipeline register. This quickly becomes unwieldy as the number of control signals grows — every new signal requires a port change on every intermediate register. Using a `typedef struct packed` instead, we define a single type that a pipeline register carries as a whole; adding a new field to the struct does not require changing the register's ports.

6.4 Next step

The next chapter covers the hazards that arise from this pipeline structure: data hazards (resolved via forwarding or stalling) and control hazards caused by resolving branches in the EX stage, which requires flushing two instructions on a misprediction.

7 Five-Stage Pipeline — Hazards

Pipeline hazards

(placeholder — fill in with week 4 content: forwarding, stall, branch flush)

8 Memory — Implementation in SystemVerilog

Memory

(placeholder — fill in with week 5 content)

9 Verification with CocoTB

Verification with CocoTB

(placeholder — fill in with week 6-7 content)

References

- IEEE. 2017. “IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language.”
- Patterson, David A., and John L. Hennessy. 2017. *Computer Organization and Design RISC-v Edition: The Hardware Software Interface*. Morgan Kaufmann.